DALEDGER · MILESTONE 1 · FOR PIKOU'S SIGN-OFF

M1 plan: schema, balance engine, login, manual entry

base: dev @ e325ceb branch: feat/m1-schema-balances Sprints 1–2 · target 3 Nov 2026

GOAL

An officer logs in on a phone, records any manual transaction with a reason, and every open device shows the new balance immediately. A transfer leaves the total held unchanged, and a reversal restores the balance. No code is written until this plan is approved.

Scope

In M1

- Project setup: Vite + React + TypeScript, Convex, Convex Auth, Tailwind + shadcn/ui, Vitest + `convex-test`.
- Full `convex/schema.ts`: all 8 tables and indexes, so later milestones don't change the schema.
- Balance engine: live balances, SG total, School receivable and committed amount, plus `balanceAsOf`.
- Officer login for a closed set of officers, with no self-signup.
- Mutations: `transactions.create` for all 7 types, `reverse`, `markWithdrawalPaid`, each writing an audit entry.
- Seed data: the 5 accounts with opening balances (test values on dev), the starting categories, and the first officer.
- Screens: Login, Home, Add transaction.

Not in M1

- Intake endpoints and refunds (M2).
- Review inbox, transaction list, filters, CSV (M3).
- Reconciliation, the stale-check flag, opening-balance correction, the negative-School warning, and the audit log viewer (M4).
- `recordPayout`, `enterPartial` and everything Naboopay (M5).
- Reports and recurring drafts (M6, M7). Their tables exist from M1 but stay empty.

Build order and files

STEP	FILES	WHAT	ADRS

1	`package.json, vite.config.ts, .env.example, tailwind setup`	Set up the project. `.env.example` lists variable names only, with no values.	–
2	`convex/schema.ts src/types.ts`	Every table from DATABASE_SCHEMA. Money fields are `v.number()`. Statuses and types are `v.union` of literals. Indexes include `by_account_occurred`, `by_to_account_occurred`, `by_status`, `by_reversal_of`, `by_source_external_kind`, `by_naboopay_ref`, `by_event` and `by_recurring_month`.	0001 0003 00
3	`convex/ledger.ts`	Pure functions with no database access: `effect(tx, accountId)` (the effect table × reversal flip × status gate), `assertAmount`, and the per-type field rules. Unit-tested on their own.	0002 0007 00
4	`convex/balances.ts`	`list`: per-account balance, total held (the 4 cash accounts only), School balance and the committed amount. `asOf(date)`: the same sum restricted to `occurredAt ≤ date`.	0004 0007 00
5	`convex/auth.config.ts, convex/auth.ts`	Convex Auth with the password provider. `currentOfficer(ctx)` fails unless the signed-in email matches an active row in `users`. Sign-up is refused for emails not in `users`.	0023
6	`convex/audit.ts`	`writeAudit(ctx, action, targetId, before`	0014

		`targetId, before, after)` , storing only the fields that changed.	
7	`convex/transactions.ts`	`create` , `reverse` , `markWithdrawalPaid` . Rules are listed below.	0002 0003 00
8	`convex/seed.ts`	An internal mutation, run once with `npx convex run` . It creates the 5 accounts, the categories (including the system ones, Unlabeled and Provider fees) and the first officer. It does nothing on a second run.	0018
9	`src/main.tsx, src/routes/Login.tsx, Home.tsx, AddTransaction.tsx, src/components/AmountInput.tsx, AccountCard.tsx, CategoryPicker.tsx, src/lib/fcfa.ts`	Phone-first screens in IBM Plex Sans. Home shows the 4 balances with their total, and School separately with the committed amount under it. The form asks for type, then amount, account, reason and category, and shows only the fields that type needs. `fcfa.ts` formats amounts as `253 650 FCFA` .	0004 0009

Rules the mutations enforce

`transactions.create`

- Requires an active officer. `recordedBy` comes from the session, never from the client.
- `amount` must be a positive integer. `reason` must not be empty.
- A category is required for income and expense, must match their direction, and can't be `Unlabeled` on a manual entry. Other types are refused if one is given.
- A transfer needs two different cash

`reverse` and `markWithdrawalPaid`

- `reverse` stores a copy of the original (same type, amount, accounts, category, event, status) with `reversalOf` set and its own reason.
- It does nothing if the target already has a reversal that is still active. A reversal can itself be reversed, which undoes it.

- A transfer needs two different cash accounts. An allocation must be on the School account. A school withdrawal goes from School to a cash account.
- A new withdrawal is `committed` with no payment date yet. Everything else is `confirmed` with `source: "manual"` and `kind: "income"`. `occurredAt` defaults to now.
- Writes a `create` audit entry.

- `markWithdrawalPaid` acts only when the status is `committed`. It sets `paid`, the destination account and `occurredAt`. A second tap does nothing.
- Each writes an audit entry (`reverse`, `mark_paid`) only when it actually acts.

Tests (required before the PR)

- **Balance engine:** every type × every status, including both legs of a transfer, a paid withdrawal versus a committed one, a reversal of each type, the exclusion of `memo`, `provisional`, `draft`, `skipped` and `committed`, School left out of the total, and `balanceAsOf` around a date boundary.
- **Guards:** reversing twice gives one reversal, and reversing the reversal restores the original. Marking paid twice moves money once.
- **Validation:** rejects zero, negative and decimal amounts, a missing reason, a wrong-direction category, a category on a transfer, and a transfer to the same account.
- **Auth:** no session, or an inactive officer, is refused.
- **Audit:** one entry per write, with only the changed fields.
- **Manual check in the browser:** two windows signed in, one adds a transaction, and the other's balance updates without a reload.

M1 acceptance, from the roadmap

CHECK	PROVEN BY
A phone entry changes the right balance on every device immediately	Two-window browser check, plus the engine tests
A transfer leaves the total held unchanged	Engine test with both legs
A reversal restores the balance	Reversal tests
The form won't save without a reason; amounts are positive whole FCFA	Validation tests, plus the form's inline error
Every write appears in the audit log	Audit tests
School is separate and left out of the SG total	Engine test, and the Home layout

Questions for Pikou

1. Sign-in method

Email and password through Convex Auth, as SECURITY.md assumes? New officers are added by inserting a `users` row with a CLI command.

2. Starting categories

Use the brief's list for now (income: Shop sales, DaBus, Sponsorship, Fundraising, Other income; expense: Supplies, Catering, Venue, Decorations, Merchandise purchase, Transport, Provider fees, Other expense) until the Treasurer confirms it?

3. Seed location

A single `convex/seed.ts` instead of the numbered `convex/migrations/000N_*.ts` files listed in DATABASE_SCHEMA? Convex applies the schema itself, so only the seed needs a file. If yes, I'll update the doc in the same PR.

4. Naboopay API version (not M1, decide before M5)

Naboopay's current docs describe API v2 and mark v1 as deprecated. Our design was checked against v1. In v2 the statuses are `paid`, `paid_and_blocked`, `cancelled` and `refunded`, with no `part_paid`. There is also a `GET /payouts` endpoint, and fees default to the merchant side. This affects ADR 0013, ADR 0015 and the M5 scope.